

Full Stack Project - Learning Management System (LMS)

Aim:

To build a full-stack Learning Management System (LMS) where students can enroll in courses, The project will involve building a Django backend, creating an API with Django Rest Framework, and integrating a React frontend, using SQLite as the database. This project demonstrates your understanding and application of Python, Django, SQL, JavaScript, and React, along with HTML and CSS for structure and styling.

Overview:

The LMS is a web application where users can log in to access educational content and track their learning progress. The system will include different user roles (students, teachers, and admins) with specific functionality tailored to each role:

- **Students:** Can browse available courses, enroll, access a list of courses they are enrolled in.
- **Teachers:** Can create and manage courses.
- **Admins:** Can create and manage courses, as well as manage users.

The project will integrate a Django backend (serving as an API for the database) with a React frontend.

The project requires implementing user authentication, secure data handling, and effective content management through a structured design. The use of SQLite as the database ensures ease of setup while maintaining all necessary relational database functionalities.

Key Elements:

- **Courses:** Courses consist of a title and description.
- **User Roles and Authentication:** Implement user registration, login, and role-based access control. The functionality that each role has access to is listed above in the overview.
- **REST API:** Develop REST endpoints using Django Rest Framework (DRF) to handle data interactions between the frontend and backend.
- **Testing:** Write automated tests for your React components and Django API methods

- **Documentation:** Include thorough documentation (README). This should include your front-end wireframes, technical explanations of how your application works under the hood, how to set up and run the application locally, and how to run the tests.

Assessment Criteria

Usability Impact:

- Design an accessible web application that follows UX/UI principles and offers a clear and intuitive user experience.
- Organize content logically, allowing for easy navigation and information retrieval.
- Write Django, React, HTML, and CSS code to create a responsive, multi-page web application with interactive functionality.
- Implement navigation, including dashboard views tailored to each user role, allowing intuitive access to course content.

Layout and Visual Impact:

- Ensure that the front-end design is consistent, accessible, and visually appealing.
- Use consistent color schemes, limited to a chosen color palette, with attention to contrast and readability.
- Include appropriate graphics and icons to enhance the user experience without distracting from key information.
- Optional: A React component library such as [Material UI](#) for front-end components such as buttons / forms etc.

Code Quality:

- Write clean, modular code that passes through standard validators (W3C for HTML, Jigsaw for CSS, linters for Python and JavaScript).
- Use Git for version control, maintaining clean a clean commit history with messages that follow set conventions.
- Ensure that the application is fully functional, with no broken links or missing assets.

Software Development Practices:

- Organize code logically in directories by function (e.g., separating backend logic, frontend components, and static files).
- Clearly document any external libraries or frameworks used, along with their sources.
- Maintain a README file that outlines the project's purpose, setup instructions, and any special considerations for deployment and testing.

Getting Started

Session with Your Mentor: Before beginning the project, schedule a session with your mentor to discuss the project requirements, break down key tasks, and set milestones for completion.

Version Control: Use Git and GitHub to manage your code, committing changes regularly with clear and informative commit messages.

Documentation: Write a README file that explains how to set up and use the project, including environment setup, database initialization, and running the server. Document API endpoints, testing procedures, and key features.

Deployment: Deploy the final project on a platform like Heroku or Netlify. Include a link to the live version in your README. Ensure that the deployment is functional, matches the local environment, and has undergone testing.

Testing: Run code validation tools and testing frameworks to ensure code quality:

- HTML: [W3C Validator](#)
- CSS: [Jigsaw Validator](#)
- React: [React Testing Library](#)
- Python: Use Django's built in test tools for integration tests (controller tests)